

Registro de uso de IA

Bitácora de las consultas realizadas a una herramienta de IA durante el desarrollo de **Portal Chat**, una SPA para conversar con personajes de Harry Potter.

Proyecto	Portal Chat (SPA vanilla JS)	Personajes	Harry, Hermione y Ron
Stack	HTML · CSS · JavaScript	APIs	HP API · Google Gemini
Despliegue	Vercel + Serverless Functions	Fecha	Julio 2026

Este documento registra cómo se utilizó la IA como herramienta de apoyo durante el proyecto: las consultas realizadas, el propósito de cada una y el criterio aplicado para evaluar y ajustar las sugerencias recibidas. Cada respuesta de la IA fue revisada, probada y adaptada manualmente antes de integrarse.

1 Definición y arquitectura del proyecto

Consultas iniciales para acotar el alcance y elegir una estructura clara.

CONSULTA

"Necesito una SPA de chat con un personaje usando IA, en vanilla JS (sin frameworks), responsive mobile-first, con routing, integración segura de la API, tests y despliegue en Vercel. ¿Cómo estructuro el proyecto?"

USO DE LA RESPUESTA

Se adoptó la estructura sugerida separando responsabilidades: `/api` para la función serverless, `/src` para la lógica (routing, chat, utils, datos) y `/tests` para las pruebas.

Criterio aplicado: se validó que cada módulo tuviera una única responsabilidad y que las funciones de datos fueran puras (sin DOM) para poder testearlas.

CONSULTA

"¿Cómo implemento routing SPA (Home, Chat, About) sin recargar la página y que funcionen los botones atrás/adelante del navegador?"

USO DE LA RESPUESTA

Se implementó un router con la **History API** (`pushState` / `popstate`). La lógica de parseo de rutas se extrajo a `router.js` como funciones puras.

Criterio aplicado: se rechazó mezclar el routing con el renderizado; se separó para poder escribir tests de `parseRoute` .

2 Integración segura con la IA (Gemini)

El núcleo del aprendizaje: no exponer la API key en el frontend.

CONSULTA

"¿Por qué no debo poner la API key de Gemini en el frontend y cómo uso una Vercel Serverless Function como proxy seguro?"

USO DE LA RESPUESTA

Se creó `api/chat.js` : el frontend envía los mensajes a esa función, que añade la API key desde `process.env.GEMINI_API_KEY` y llama a Gemini. La clave nunca llega al navegador.

Criterio aplicado: se comprendió el riesgo (una key en el cliente es visible y robable) y se verificó que la key solo vive en el servidor y en las variables de entorno de Vercel.

CONSULTA

"¿Qué debe contener el archivo `.env` para no comprometer mis datos, y qué va en `.env.example` y `.gitignore`?"

USO DE LA RESPUESTA

Se configuró `.env.local` (clave real, ignorada por git), `.env.example` (solo nombres, versionado) y un `.gitignore` que excluye los archivos de entorno.

Criterio aplicado: se confirmó con `git status` que la clave real nunca se sube al repositorio.

CONSULTA

"Diseña los system prompts para que cada personaje responda en su personalidad, en español y con respuestas cortas tipo chat."

USO DE LA RESPUESTA

Se redactó un *system prompt* único por personaje en `characters.js`, definiendo personalidad, forma de hablar, límites y estilo (1–3 frases).

Criterio aplicado: se iteró el tono probando respuestas y se añadieron reglas explícitas: no romper el personaje, no admitir ser una IA y mantener contenido apto para todo público.

3 Consumo de API de datos y manejo de estados

Fetch, transformación de JSON y estados loading / success / error.

CONSULTA

"Quiero traer los personajes desde una API pública. Probamos varias (Rick and Morty, Disney) hasta usar la HP API. ¿Cómo separo el fetching del parseo?"

USO DE LA RESPUESTA

Se creó `hpapi.js` con `fetchAllCharacters` (petición) y `parseCharacter` / `findCharacter` (transformación pura). La UI maneja estados `loading`, `success` y `error` por personaje, con avatar de respaldo.

Criterio aplicado: se descartó la Disney API tras comprobar que no devolvía a los personajes buscados; se evaluó y eligió la HP API por traer imágenes y datos fiables. La IA no impuso la fuente: se verificó con pruebas reales.

CONSULTA

"¿Cómo transformo el historial de mensajes al formato que espera Gemini y cómo parseo su respuesta de forma segura?"

USO DE LA RESPUESTA

Se implementaron `toGeminiContents` (mapea roles a `user` / `model`) y `parseGeminiResponse` (extrae el texto y lanza error si la estructura es inválida) en `utils.js`.

Criterio aplicado: se añadió manejo de errores en vez de asumir que la respuesta siempre trae texto.

4 Pruebas y despliegue

Tests con Vitest (mock de fetch) y publicación en Vercel.

CONSULTA

"Escribe tests unitarios con Vitest mockeando fetch para probar sin red: transformación de datos, parseo, routing y la petición al backend."

USO DE LA RESPUESTA

Se escribieron pruebas en `tests/` cubriendo `utils`, `chat` (con `fetch` mockeado), `hpapi` y `router`, incluyendo casos de error (JSON corrupto, respuesta sin texto, fetch fallido).

Criterio aplicado: se revisó cada aserción manualmente y se añadieron casos borde que la IA no había contemplado inicialmente.

CONSULTA

"Está desplegado en Vercel pero la API de Gemini no funciona. ¿Cómo la conecto y diagnostico el error?"

USO DE LA RESPUESTA

Se siguió una guía de diagnóstico: verificar que `GEMINI_API_KEY` esté en las Environment Variables, **redesplegar** tras añadirla, revisar la pestaña Network para leer el error real y comprobar la función abriendo `/api/chat`.

Criterio aplicado: se entendió que Vercel solo inyecta las variables de entorno en el momento del deploy, por lo que un cambio de variable exige un nuevo despliegue.

5 Herramientas y modelos utilizados

HERRAMIENTA	USO EN EL PROYECTO
Asistente de IA (chat)	Arquitectura, generación y revisión de código, depuración y documentación.
Google Gemini API	Motor conversacional de los personajes (en producción, vía proxy).
HP API	Fuente de imágenes y datos reales de los personajes.
Vitest	Framework de pruebas unitarias.
Vercel	Hosting y funciones serverless.

Reflexión sobre el uso de IA. La IA se utilizó como apoyo para acelerar el desarrollo y aprender conceptos (proxy seguro, History API, mock de fetch), no como reemplazo del criterio propio. Cada sugerencia se probó, se cuestionó y se ajustó: se cambió la fuente de datos cuando una API no servía, se añadieron casos de prueba faltantes y se verificó la seguridad de la API key. El resultado final es comprendido y defendible línea por línea.